From EF Core 7.0, we can use the **ExecuteUpdate**() and **ExecuteDelete**() methods to update or delete data. Note that these two methods **do not involve the entity tracking feature**. So, once you call these two methods, the changes will be executed immediately. There is no need to call the **SaveChanges**() method.

****

ExecuteUpdate()

The `ExecuteUpdate()` method is used to update data without loading the entities from the database. You can use it to update one or more entities by adding the `Where()` clause.

For example, we want to update the status of the invoices that were created before a specific date. The code is as follows:

```
[HttpPut]
[Route("status/overdue")]
public async Task<ActionResult> UpdateInvoicesStatusAsOverdue(DateTime
date)
{
    var result = await context.Invoices
        .Where(i => i.InvoiceDate < date && i.Status == InvoiceStatus.
AwaitPayment)
        .ExecuteUpdateAsync(s => s.SetProperty(x => x.Status,
InvoiceStatus.Overdue));
```

****

This query can update multiple invoices at the same time. It does benefit from the strong type support but has the same efficiency as the raw SQL query. If we need to update more than one property, you can use the SetProperty() method multiple times:

```
var result = await context.Invoices
        .Where(i => i.InvoiceDate < date && i.Status == InvoiceStatus.
AwaitPayment)
        .ExecuteUpdateAsync(s =>
            s.SetProperty(x => x.Status, InvoiceStatus.Overdue)
            .SetProperty(x => x.LastUpdatedDate, DateTime.Now));
```

In addition, the `Where()` clause can reference the other entities. So, the `ExecuteUpdate()` method is always recommended to update multiple entities, instead of using the raw SQL query.

****

```
await context.Invoices.Where(x => x.InvoiceDate < date).
ExecuteDeleteAsync();
```

****

An API endpoint can be called by multiple clients at the same time. If the endpoint updates data, the data may be updated by another client before the current client completes the update. When the same entity is updated by multiple clients, it can cause a concurrency conflict, which may result in data loss or inconsistency, or even cause data corruption.

****

There are two ways to handle concurrency conflicts:

- **Pessimistic concurrency control**: This uses database locks to prevent multiple clients from updating the same entity at the same time. When a client tries to update an entity, it will first acquire a lock on the entity. If the lock is acquired successfully, only this client can update the entity, and all other clients will be blocked from updating the entity until the lock is released. However, this approach may result in performance issues when the number of concurrent clients is large because managing locks is expensive. EF Core does not have built-in support for pessimistic concurrency control.
- **Optimistic concurrency control**: This way does not involve locks; instead, a version column is used to detect concurrency conflicts. When a client tries to update an entity, it will first get the value of the version column, and then compare this value with the old value when updating the entity. If the value of the version column is the same, it means that no other client has updated the entity. In this case, the client can update the entity. But if the value of the version column is different from the old value, it means that another client has updated the entity. In this case, EF Core will throw an exception to indicate the concurrency conflict. The client can then handle the exception and retry the update operation.

****

One of problems when handling multiple requests in the same time:

```
[HttpPost("{id}/sell/{quantity}")]
public async Task<ActionResult<Product>> SellProduct(int id, int
```

a Access in ASP.NET Core (Part 3: Tips)

```
quantity, int delay = 0)
{
    // Omitted code for brevity
    await Task.Delay(TimeSpan.FromSeconds(delay));
    product.Inventory -= quantity;
    await context.SaveChangesAsync();
    return product;
}
```

****

3. Send the first request and then send the second request in `2` seconds. The expected result should be that the first request will succeed and the second request will fail. But actually, both requests will succeed. The responses show that the `Inventory` property of the product is updated to `5`, which is incorrect. The initial value of the `Inventory` property is `15`, and we sold 20 products, so how can the `Inventory` property be updated to `5`?

Let us see what happens in the application:

1. Client A calls the API endpoint to sell a product and wants to sell 10 products.
2. Client A checks the `Inventory` property and finds that the number of products in stock is 15, which is enough to sell.
3. Almost at the same time, client B calls the API endpoint to sell a product and wants to sell 10 products.
4. Client B checks the `Inventory` property and finds that the number of products in stock is 15 because client A has not updated the `Inventory` property yet.
5. Client A subtracts 10 from the `Inventory` property, which results in a value of `5`, and saves the changes to the database. Now, the number of products in stock is `5`.
6. Client B also subtracts 10 from the `Inventory` property and saves the changes to the database. The problem is that the number of products in stock has been updated to `5` by client A, but client B does not know this. So, client B also updates the `Inventory` property to `5`, which is incorrect.

****

To solve this problem, EF Core provides optimistic concurrency control. There are two ways to use optimistic concurrency control:

- Native database-generated concurrency token
- Application-managed concurrency token

****

Native database-generated concurrency token

Some databases, such as SQL Server, provide a native mechanism to handle concurrency conflicts. To use the native database-generated concurrency token in SQL Server, we need to create a new property for the `Product` class and add a `[Timestamp]` attribute to it. The following code shows the updated `Product` class:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Inventory { get; set; }
    // Add a new property as the concurrency token
    public byte[] RowVersion { get; set; }
}
```

In the Fluent API configuration, we need to add the following code to map the `RowVersion` property to the `rowversion` column in the database:

```
modelBuilder.Entity<Product>()
    .Property(p => p.RowVersion)
    .IsRowVersion();
```


****

If you prefer to use the data annotation configuration, you can add the `[Timestamp]` attribute to the `RowVersion` property, and EF Core will automatically map it to the `rowversion` column in the database, as shown in the following code:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Inventory { get; set; }
    // Add a new property as the concurrency token
    [Timestamp]
    public byte[] RowVersion { get; set; }
}
```

****

By using concurrency control, EF Core not only checks the ID of the entity but also checks the value of the rowversion column. If the value of the rowversion column is not the same as the value in the database, it means that the entity has been updated by another client, and the current update operation should be aborted.

****

Note that the rowversion column type is available for SQL Server, but not for other databases, such as SQLite. Different databases may have different types of concurrency tokens, or may not support the concurrency token at all.

****

Application-managed concurrency token

If the database does not support the built-in concurrency token, we can manually manage the concurrency token in the application. Instead of using the rowversion column, which can be automatically updated by the database, we can use a property in the entity class to manage the concurrency token and assign a new value to it every time the entity is updated.

Here is an example of using the application-managed concurrency token:

1. First, we need to add a new property to the `Product` class, as shown in the following code:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Inventory { get; set; }
    // Add a new property as the concurrency token
    public Guid Version { get; set; }
}
```

2. Update the Fluent API configuration to specify the `Version` property as the concurrency token:

```
modelBuilder.Entity<Product>()
    .Property(p => p.Version)
    .IsConcurrencyToken();
```


****

3. The corresponding data annotation configuration is as follows:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Inventory { get; set; }
    // Add a new property as the concurrency token
    [ConcurrencyCheck]
    public Guid Version { get; set; }
}
```

4. Because this `Version` property is not managed by the database, we need to manually assign a new value whenever the entity is updated.

The following code shows how to update the `Version` property when the entity is updated:

```
[HttpPost("{id}/sell/{quantity}")]
public async Task<ActionResult<Product>> SellProduct(int id, int
quantity)
{
    // Omitted for brevity.
    product.Inventory -= quantity;
    // Manually assign a new value to the Version property.
    product.Version = Guid.NewGuid();
    await context.SaveChangesAsync();
    return product;
}
```

****

Sometimes we need to work with an existing database. In this case, we need to create the entity classes and DbContext from the existing database schema. This is called database-first or reverse engineering. This is useful when we want to migrate an existing application to EF Core.

****

1. First, let us create a new web API project. Run the following command in the terminal:

```
dotnet new webapi -n EfCoreReverseEngineeringDemo
```

2. To use reverse engineering, we need to install the `Microsoft.EntityFrameworkCore.Design` NuGet package. Navigate to the `EfCoreReverseEngineeringDemo` folder, and run the following command in the terminal to install it:

```
dotnet add package Microsoft.EntityFrameworkCore.Design
```

3. Then, add the `Microsoft.EntityFrameworkCore.SqlServer` NuGet package to the project:

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

   If you use other database providers, such as SQLite, you need to install the corresponding NuGet package. For example, to use SQLite, you need to install the `Microsoft.EntityFrameworkCore.Sqlite` NuGet package. You can find the list of supported database providers at `https://learn.microsoft.com/en-us/ef/core/providers/`.

4. Next, we will use the `dbcontext scaffold` command to generate the entity classes and `DbContext` from the database schema. This command needs the connection string of the database and the name of the database provider. We can run the following command in the terminal:

```
dotnet ef dbcontext scaffold "Server=(localdb)\
mssqllocaldb;Initial Catalog=EfCoreRelationshipsDemoDb;Trusted_
Connection=True;" Microsoft.EntityFrameworkCore.SqlServer
```

   By default, the generated files will be placed in the current directory. We can use the `--context-dir` and `--output-dir` options to specify the output directory of the `DbContext` and entity classes. For example, we can run the following command to generate the `DbContext` and entity classes in the `Data` and `Models` folders, respectively:

****

```
dotnet ef dbcontext scaffold "Server=(localdb)\
mssqllocaldb;Initial Catalog=EfCoreRelationshipsDemoDb;Trusted_
Connection=True;" Microsoft.EntityFrameworkCore.SqlServer
--context-dir Data --output-dir Models
```

The default name of the `DbContext` class will be the same as the database name, such as `EfCoreRelationshipsDemoDbContext.cs`. We can also change the name of the `DbContext` class by using the `--context` option. For example, we can run the following command to change the name of the `DbContext` class to `AppDbContext`:

```
dotnet ef dbcontext scaffold "Server=(localdb)\
```



```
mssqllocaldb;Initial Catalog=EfCoreRelationshipsDemoDb;Trusted_
Connection=True;" Microsoft.EntityFrameworkCore.SqlServer
--context-dir Data --output-dir Models --context AppDbContext
```

If the command executes successfully, we will see the generated files in the `Data` and `Models` folders.

****

5. Open the `AppDbContext.cs` file, and we will see a warning in the following code:

```
protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
#warning To protect potentially sensitive information in your
connection string, you should move it out of source code.
You can avoid scaffolding the connection string by using the
Name= syntax to read it from configuration - see https://
go.microsoft.com/fwlink/?linkid=2131148. For more guidance
on storing connection strings, see http://go.microsoft.com/
fwlink/?LinkId=723263.
        => optionsBuilder.UseSqlServer("Server=(localdb)\\
mssqllocaldb;Initial Catalog=EfCoreRelationshipsDemoDb;Trusted_
Connection=True;");
```

This warning tells us that we should not store the connection string in the source code. Instead, we should store it in a configuration file, such as `appsettings.json`.

****

Some models or properties may not be represented correctly in the database. For example, if your models have inheritance, the generated code will not include the base class because the base class is not represented in the database. Also, some column types may not be able to be mapped to the corresponding CLR types. For example, the Status column in the Invoice table is of the nvarchar(16) type, which will be mapped to the string type in the generated code, instead of the Status enum type.

****